

Property-Based Testing

Guillermo Polito

guillermo.polito@inria.fr
@guillep



inria



Université
de Lille

Goals

- Combining Fuzzing (generators) and general user-defined oracles (properties)
- General properties vs specific assertions
- Library examples

Properties

- **Developers define function *properties***
- Example

`InsertIsSorted x xs = ordered xs ==> ordered (insert x xs)`

If a list is sorted, inserting an element should keep it sorted

Properties

- Developers define function *properties*
- Example

The texted function

`InsertIsSorted x xs = ordered xs ==> ordered (insert x xs)`

If a list is sorted, inserting an element should keep it sorted

Properties

- Developers define function *properties*
- Example

Implication to discern inputs

InsertIsSorted x xs = ordered xs ==> ordered (insert x xs)

*For every a list that is sorted, **then** inserting an element should keep it sorted*

$$\forall list, x / sorted(list) \Rightarrow sorted(insert(x, list))$$

Generators

- How do we test *every sorted list*, and *every x*?

$$\forall \text{ list, } x / \text{sorted (list)} \Rightarrow \text{sorted}(\text{insert}(x, \text{list}))$$

Generators

- How do we test *every sorted list*, and *every x*?

$$\forall \text{ list, } x / \text{sorted (list)} \Rightarrow \text{sorted(insert(x, list))}$$

- We don't: fuzzers are bound e.g., # of inputs or time

Simple Generators

- Generators/Fuzzers are type guided

```
InsertIsSorted :: Int -> [Int] -> Bool
```

```
InsertIsSorted x xs = ordered xs ==> ordered (insert x xs)
```

- The type signature tells what to generate
- Random: Difficult to explore border cases

Defining Generators

- Need ****custom data types****
- See example below with Dice

```
import Test.QuickCheck
```

```
data Dice = Dice Int
    deriving (Show)
```

```
prop_DiceLowerBounds :: Dice -> Bool
prop_DiceLowerBounds (Dice faces) =
    faces > 0
```

```
instance Arbitrary (Dice) where
    arbitrary = do
        t <- choose (0, 6)
        return (Dice t)
```

```
main = do
    quickCheck prop_DiceLowerBounds
```

Some Property-based Testing Frameworks

- <https://jqwik.net/> — Java + Kotlin
- https://www.scalatest.org/user_guide/property_based_testing - Scala
- <https://hypothesis.readthedocs.io/> — Python
- <https://github.com/ohbarye/pbt> — Ruby
- <https://github.com/silentbicycle/theft> - C
- <https://crates.io/crates/quickcheck> — Rust
- ...

Conclusion

- Property based testing is a good balance between fuzzing and testing
- Generic properties are sometimes hard to express
- Generic fuzzers are sometimes ineffective and need some tuning